

ALBERI AVL

SONO DEGLI ALBERI BINARI DI RICERCA (ABR) CHE RISPETTANO LA SEGUENTE PROPRIETÀ:

$$\text{SIA } T \text{ UN ABR, } \forall x \in T (|h(x \rightarrow \text{left}) - h(x \rightarrow \text{right})| \leq 1)$$

CIOÈ, PER OGNI NODO DI T L'ALTEZZA IN VALORE ASSOLUTO DEI SOTTOALBERI S_x E D_x DIFFERISCE AL PIÙ DI 1

AVL MINIMI

LA CLASSE DI AVL DI ALTEZZA h CHE POSSIEME IL MINIMO NUMERO DI NODI.

RAPPRESENTANO IL CASO PRESTAZIONALE PEGGIORE, PERCHÉ VEDREMO CHE IL COSTO DEGLI ALGORITMI DIPENDE DALL'ALTEZZA, QUINDI AVERE MENO DATI A PARITÀ DI ALTEZZA RIDUCE L'EFFICIENZA DELLA STRUTTURA. DIMOSTRARE UN LIMITE ASINTOTICO PER QUESTA CLASSE GARANTISCE LA VALIDITÀ PER TUTTI GLI ALBERI AVL.

$$\begin{cases} N_h = 1 + N_{h-1} + N_{h-2} \\ N_{-1} = 0 \\ N_0 = 1 \end{cases} \quad N_h: \text{NUMERO DI NODI DI UN ALBERO ALTO } h$$

$$\text{INOLTRE } N_h = F_{h+3} - 1$$

NODO DI UN ALBERO AVL

KEY
h
LEFT RIGHT

IL CAMPO h INDICA L'ALTEZZA DELL'ALBERO, CHE RICHIEDEREBBE ALTAMENTE UN ALGORITMO CON TEMPO* LINEARE RISPETTO AL NUMERO DEI NODI DELL'ALBERO

*DI ESECUZIONE

SI OTTIENE QUINDI L'ALGORITMO $ALTEZZA(T)$ CHE RITORNA L'ALTEZZA DELL'ALBERO CON UN COSTO COMPUTAZIONALE COSTANTE

ALTEZZA(h)

IF $T == \text{NIL}$

RETURN -1

ELSE

RETURN $T \rightarrow h$

INSERIMENTO IN UN ALBERO AVL

L'ALGORITMO SI DIVIDE IN 2 FASI:

1. INSERIMENTO SECONDO LE REGOLE DI UN ABR
2. AGGIORNAMENTO ALTEZZE CON EVENTUALE BILANCIAMENTO, IN CASO DI VIOLAZIONE DELLA PROPRIETÀ DEGLI AVL (ROTAZIONI)

INSERT-AVL(T, k)

IF $T \neq \text{NIL}$

IF $T \rightarrow \text{KEY} < k$

$T \rightarrow \text{RIGHT} = \text{INSERT-AVL}(T \rightarrow \text{right}, k)$

$T = \text{BILANCIA-DX}(T)$

ELSE IF $T \rightarrow \text{KEY} > k$

$T \rightarrow \text{LEFT} = \text{INSERT-AVL}(T \rightarrow \text{left}, k)$

$T = \text{BILANCIA-SX}(T)$

ELSE

$T = \text{ALLOCA-NODO-AVL}$

$T \rightarrow \text{KEY} = k$

$T \rightarrow \text{LEFT} = \text{NIL}$

$T \rightarrow \text{RIGHT} = \text{NIL}$

$T \rightarrow h = 0$

RETURN T

BILANCIAMENTO IN UN ALBERO AVL

GLI ALGORITMI DI BILANCIAMENTO HANNO IL COMPITO DI ANALIZZARE LO STATO DEL NODO T DOPO GLI INSERIMENTI O LE CANCELLAZIONI, IN 3 PASSI:

1. CALCOLO DELLA DIFFERENZA DI ALTEZZA DEI SOTTOALBERI DI T
2. SE LA DIFFERENZA NON INDICA UNA VIOLAZIONE DELLE PROPRIETÀ, L'ALGORITMO SI LIMITA AD AGGIORNARE LE ALTEZZE DI T
3. IN CASO DI VIOLAZIONE, ANALIZZA LA "PENDENZA"* PER SCEGLIERE LA ROTAZIONE CORRETTA DA APPLICARE

* DIFFERENZA DI ALTEZZA TRA SOTTOALBERO SX E DX

BILANCIA-DX(T)

IF T $\neq$ NIL

IF ALTEZZA(T $\rightarrow$ right) - ALTEZZA(T $\rightarrow$ left) > 1

DX = T $\rightarrow$ RIGHT

IF ALTEZZA(DX $\rightarrow$ RIGHT) $\geq$ ALTEZZA(DX $\rightarrow$ LEFT)

T = ROTAZIONE-DX(T)

ELSE

T = ROTAZIONE-DOPPIA-DX(T)

ELSE

T $\rightarrow$ h = max (ALTEZZA(T $\rightarrow$ left), ALTEZZA(T $\rightarrow$ right)) + 1

RETURN T

BILANCIA-SX (T)

IF $T \neq \text{NIL}$

IF $\text{ALTEZZA}(T \rightarrow \text{LEFT}) - \text{ALTEZZA}(T \rightarrow \text{RIGHT}) > 1$

$SX = T \rightarrow \text{LEFT}$

IF $\text{ALTEZZA}(SX \rightarrow \text{LEFT}) \geq \text{ALTEZZA}(SX \rightarrow \text{RIGHT})$

$T = \text{ROTAZIONE-SX}(T)$

ELSE

$T = \text{ROTAZIONE-DOPPIA-SX}(T)$

ELSE

$T \rightarrow h = \max(\text{ALTEZZA}(T \rightarrow \text{LEFT}), \text{ALTEZZA}(T \rightarrow \text{RIGHT})) + 1$

RETURN T

ROTAZIONI IN UN ALBERO AVL

LE ROTAZIONI SONO DELLE OPERAZIONI DI RIORGANIZZAZIONE STRUTTURALE DEI NODI, HANNO LO SCOPO DI RIPRISTINARE LA PROPRIETÀ DELL'ALTEZZA DEGLI AVL A SEGUITO DI UN INSERIMENTO O UNA CANCELLAZIONE.

NE ESISTONO DI 2 TIPI:

1. ROTAZIONI SINGOLE: RISOLVONO SBILANCIAMENTI IN CUI IL SOTTOALBERO CRITICO PENDE VERSO L'ESTERNO (LINEARE)
2. ROTAZIONI DOPPIE: RISOLVONO SBILANCIAMENTI IN CUI IL SOTTOALBERO CRITICO PENDE VERSO L'INTERNO (ZIG-ZAG)

ROTAZIONI SINGOLE

ROTAZIONE-SX(T)

NEW T = T → LEFT

T → LEFT = NEW T → RIGHT

NEW T → RIGHT = T

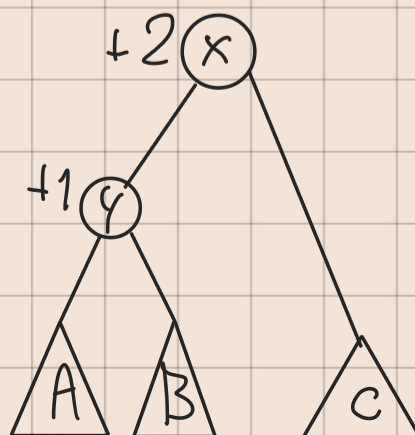
T → h = $\max(\text{ALTEZZA}(T \rightarrow \text{LEFT}), \text{ALTEZZA}(T \rightarrow \text{RIGHT})) + 1$

NEW T → h = $\max(\text{ALTEZZA}(\text{NEW T} \rightarrow \text{LEFT}), \text{ALTEZZA}(\text{NEW T} \rightarrow \text{RIGHT})) + 1$

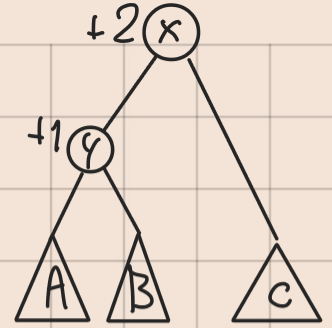
return NEW T

ESEMPIO

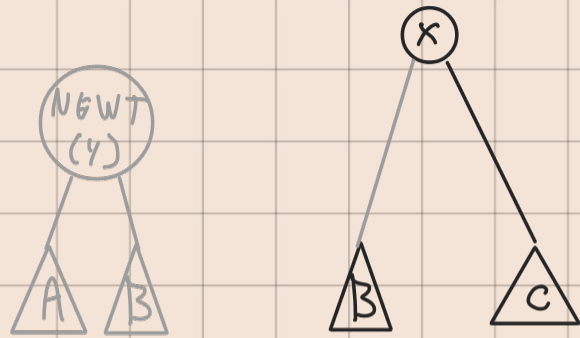
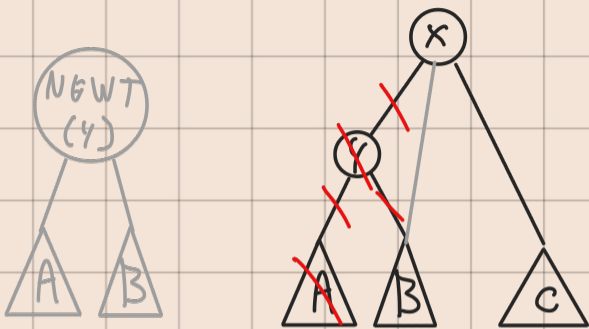
ROTAZIONE SX DOPO UN INSERIMENTO IN B



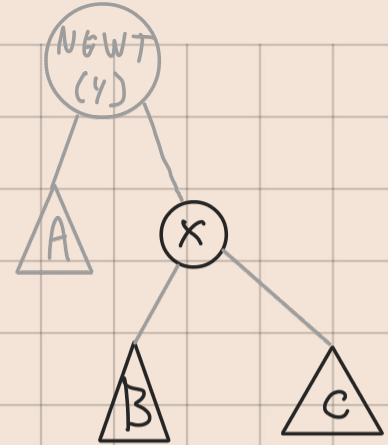
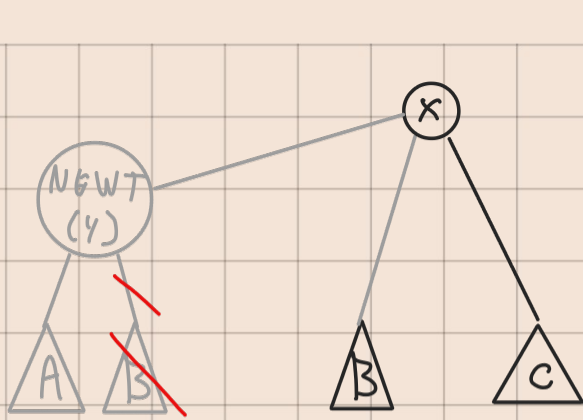
ROTATE-SK (T) // $T = x$
 • NEWT = $T \rightarrow \text{LEFT}$ // NEWT = y
 $T \rightarrow \text{LEFT} = \text{NEWT} \rightarrow \text{RIGHT}$
 $\text{NEWT} \rightarrow \text{RIGHT} = T$
 $T \rightarrow h = \max(\text{alterar}(T \rightarrow \text{left}), \text{alterar}(T \rightarrow \text{right})) + 1$
 $\text{newT} \rightarrow = \max(\text{alterar}(\text{newT} \rightarrow \text{left}), \text{alterar}(\text{newT} \rightarrow \text{right})) + 1$
 return newT



ROTATE-SK (T) // $T = x$
 NEWT = $T \rightarrow \text{LEFT}$ // NEWT = y
 • $T \rightarrow \text{LEFT} = \text{NEWT} \rightarrow \text{RIGHT}$ // $x \rightarrow \text{LEFT} = B$
 $\text{NEWT} \rightarrow \text{RIGHT} = T$
 $T \rightarrow h = \max(\text{alterar}(T \rightarrow \text{left}), \text{alterar}(T \rightarrow \text{right})) + 1$
 $\text{newT} \rightarrow = \max(\text{alterar}(\text{newT} \rightarrow \text{left}), \text{alterar}(\text{newT} \rightarrow \text{right})) + 1$
 return newT



ROTATE-SK (T) // $T = x$
 NEWT = $T \rightarrow \text{LEFT}$ // NEWT = y
 $T \rightarrow \text{LEFT} = \text{NEWT} \rightarrow \text{RIGHT}$ // $x \rightarrow \text{LEFT} = B$
 • $\text{NEWT} \rightarrow \text{RIGHT} = T$ // $\text{NEWT} \rightarrow \text{RIGHT} = x$
 $T \rightarrow h = \max(\text{alterar}(T \rightarrow \text{left}), \text{alterar}(T \rightarrow \text{right})) + 1$
 $\text{newT} \rightarrow = \max(\text{alterar}(\text{newT} \rightarrow \text{left}), \text{alterar}(\text{newT} \rightarrow \text{right})) + 1$
 return newT



T

SISTEMA LE ALTEZZE (PRIMA QUELLA DI X, PERCHÉ LA NUOVA ALTEZZA DI ^{NEWT}Y DIPENDE DA ESSA) E RITORNO LA RADICE DEL NUOVO ALBERO

ROTAZIONE DX (T)

$NEWT = T \rightarrow RIGHT$

$T \rightarrow RIGHT = NEWT \rightarrow LEFT$

$NEWT \rightarrow LEFT = T$

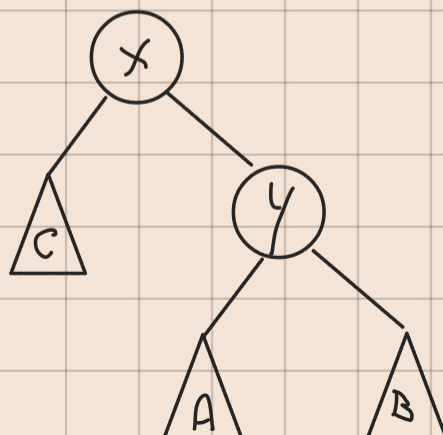
$T \rightarrow h = \max(\text{altezza}(T \rightarrow LEFT), \text{altezza}(T \rightarrow RIGHT)) + 1$

$NEWT \rightarrow h = \max(\text{altezza}(NEWT \rightarrow LEFT), \text{altezza}(T \rightarrow RIGHT)) + 1$

RETURN NEWT

ESEMPIO

ROTAZIONE DX DOPO UN INSERIMENTO IN B



ROTATION E-DX (T) // T = x

• NEW T = T → RIGHT // NEW T = y

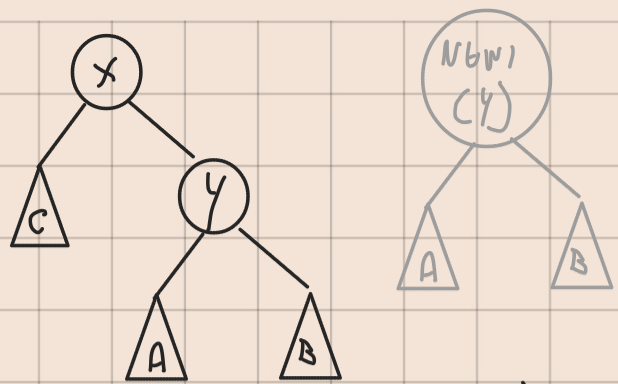
T → RIGHT = NEW T → LEFT

NEW T → LEFT = T

T → h = max (altree (T → LEFT), altree (T → RIGHT)) + 1

NEW T → h = max (altree (NEW T → LEFT), altree (T → RIGHT)) + 1

RETURN NEW T



ROTATION E-DX (T) // T = x

NEW T = T → RIGHT // NEW T = y

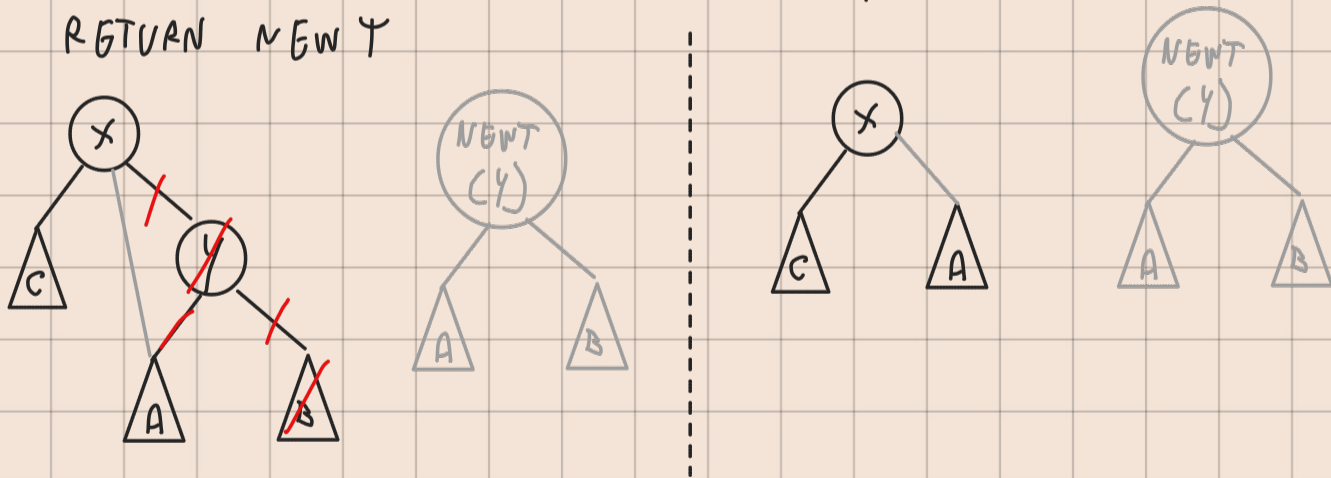
• T → RIGHT = NEW T → LEFT // x → RIGHT = A

NEW T → LEFT = T

T → h = max (altree (T → LEFT), altree (T → RIGHT)) + 1

NEW T → h = max (altree (NEW T → LEFT), altree (T → RIGHT)) + 1

RETURN NEW T



ROTATION E-DX (T) // T = x

NEW T = T → RIGHT // NEW T = y

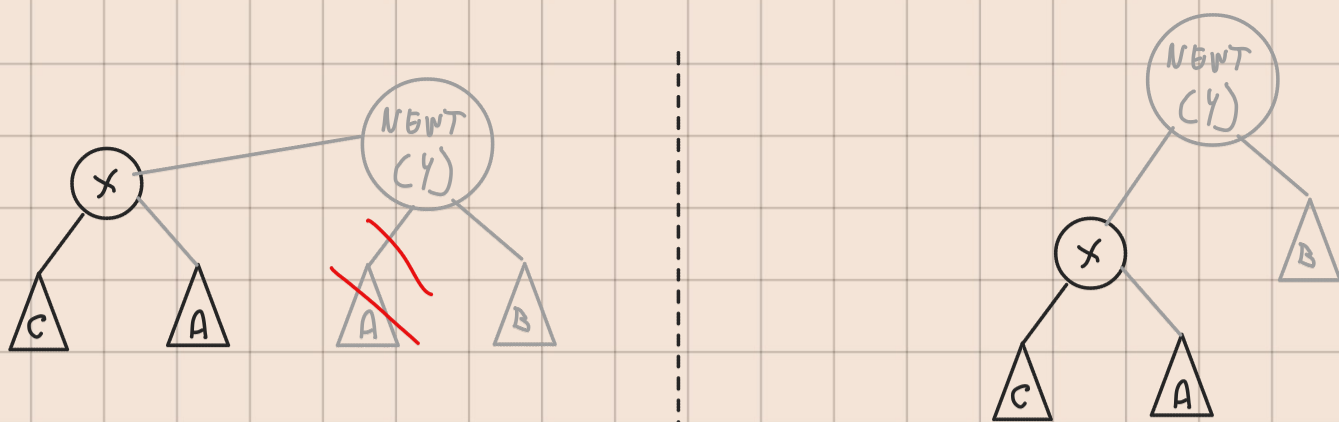
T → RIGHT = NEW T → LEFT // x → RIGHT = A

• NEW T → LEFT = T // NEW T → LEFT = x

T → h = max (altree (T → LEFT), altree (T → RIGHT)) + 1

NEW T → h = max (altree (NEW T → LEFT), altree (T → RIGHT)) + 1

RETURN NEW T

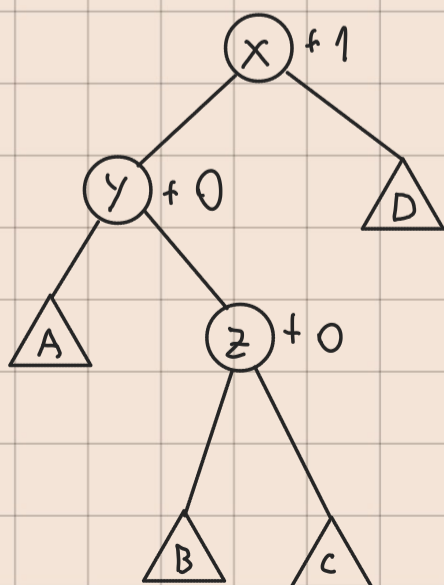


SISTEMA LE ALTEZZE (PRIMA QUELLA DI X, PERCHÉ LA NUOVA ALTEZZA DI ^{NEW}Y DIPENDE DA ESSA) E RITORNO LA RADICE DEL NUOVO ALBERO

ROTAZIONI DOPPIE

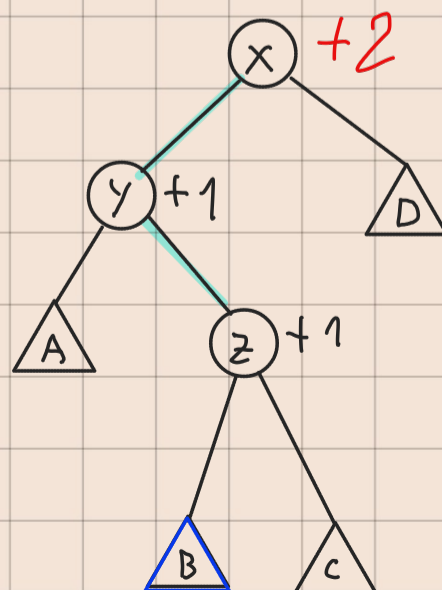
ROTAZIONE DOPPIA SX

SI UTILIZZA QUANDO L'ALBERO È SBILANCIATO A SX, MA IL RAMO HA UNA FORMA A **ZIG-ZAG**.
CIOÈ, IL NODO CRITICO HA ALTEZZA MAGGIORE A SINISTRA, MA IL SUO FIGLIO SINISTRO HA UN SOTTOALBERO DESTRO PIÙ PROFONDO



AVL BILANCIATO

INSERIMENTO IN B



ALBERO SBILANCIATO

L'OBIETTIVO È OTTENERE UN NUOVO AVL, MA UNA ROTAZIONE SINGOLA ANDREBBE SOLO A CAMBIARE IL GENITORE DEL SOTTOALBERO CRITICO, LASCIANDO L'ALBERO SBILANCIATO.

LA DOPPIA ROTAZIONE EFFETTUA 2 OPERAZIONI:

1. ALLINEAMENTO: EFFETTIAMO UNA ROTAZIONE-DX SUL FIGLIO (Y), "RADDIRIZZANDO" LO ZIG-ZAG

ROTAZIONE-DX (T) // $T = Y$

$NEW T = T \rightarrow RIGHT$ // $NEW T = Z$

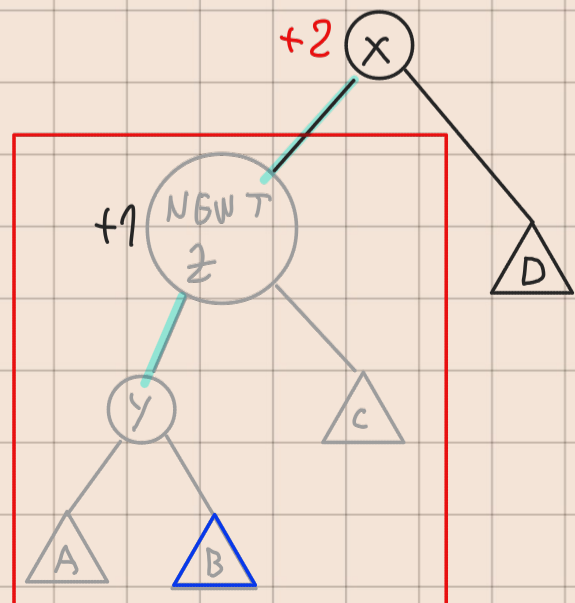
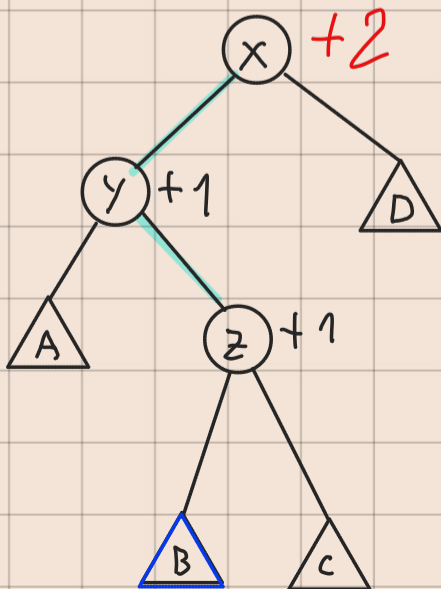
$T \rightarrow RIGHT = NEW T \rightarrow LEFT$ // $Y \rightarrow RIGHT = B$

$NEW T \rightarrow LEFT = T$ // $NEW T \rightarrow LEFT = Y$

$T \rightarrow h = \max(\text{altezza}(T \rightarrow LEFT), \text{altezza}(T \rightarrow RIGHT)) + 1$

$NEW T \rightarrow h = \max(\text{altezza}(NEW T \rightarrow LEFT), \text{altezza}(T \rightarrow RIGHT)) + 1$

RETURN NEW T



2. BILANCIAMENTO DEFINITIVO: OAA BASTA FARE UNA ROTAZIONE SINISTRA SEMPLICE SULLA RADICE, CHE ACCORCIA L'ALTEZZA TOTALE

ROTAZIONE-SX(T) // $T = x$

newT = T → LEFT // newT = z

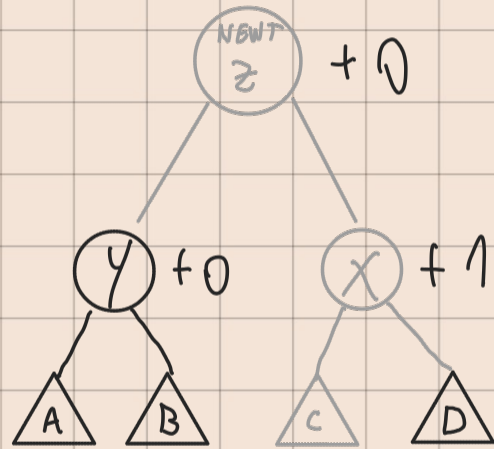
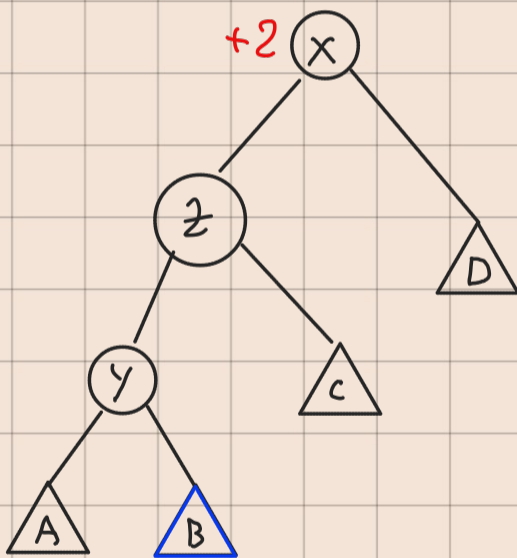
T → LEFT = newT → RIGHT // $x \rightarrow \text{LEFT} = c$

newT → RIGHT = T // ~~newT → RIGHT = x~~

$T \rightarrow h = \max(\text{altezza}(T \rightarrow \text{left}), \text{altezza}(T \rightarrow \text{right})) + 1$

$\text{newT} \rightarrow = \max(\text{altezza}(\text{newT} \rightarrow \text{left}), \text{altezza}(\text{newT} \rightarrow \text{right})) + 1$

return newT



SI OTTIENE COSÌ L'ALGORITMO ROTAZIONE-DOPPIA-SX(T)

ROTAZIONE-DOPPIA-SX(T)

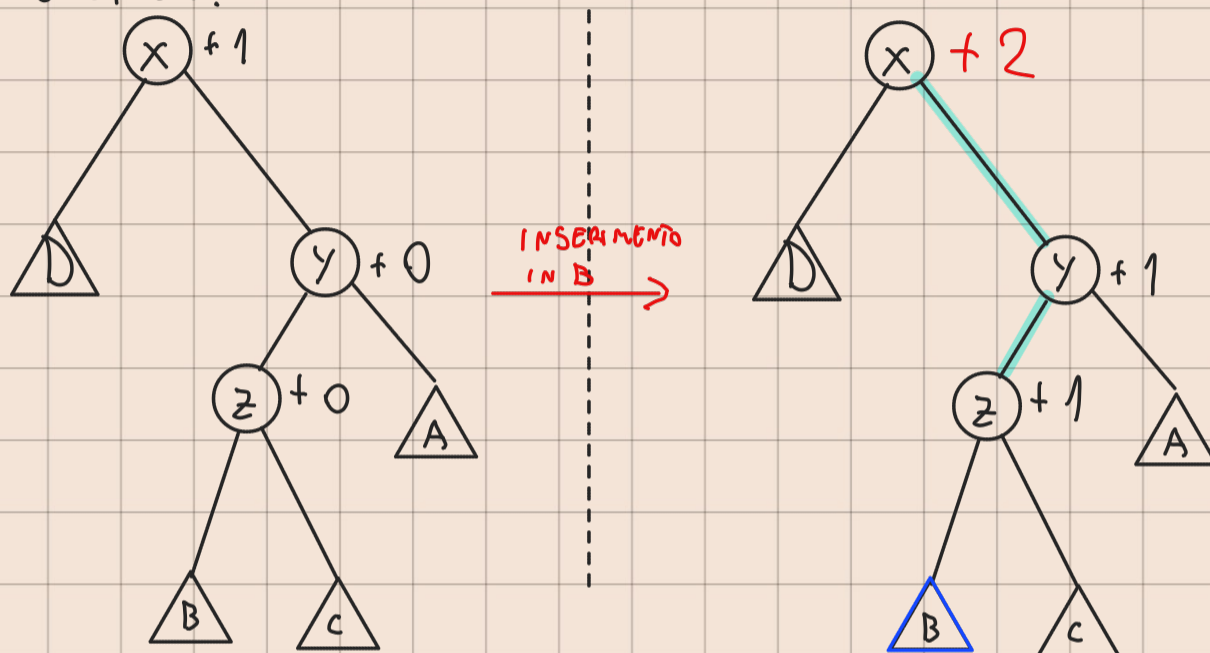
T → LEFT = ROTAZIONE-DESTRA(T → LEFT)

T = ROTAZIONE-SINISTRA(T)

RETURN T

ROTAZIONE-DOPPIA-DX

SI UTILIZZA QUANDO L'ALBERO È SBILANCIATO A DX, MA IL RAMO HA UNA FORMA A ZIG-ZAG. CIOÈ, IL NODO CRITICO HA ALTEZZA MAGGIORE A DESTRA MA IL SUO FIGLIO DESTRO HA UN SOTTOALBERO SINISTRO PIÙ PROFONDO.



EFFETTUA LE OPERAZIONI:

1. ALLINEAMENTO: SI ESEGUE UNA ROTAZIONE-SX SU FIGLIO (Y) RADDIREZZANDO LO ZIG-ZAG

ROTAZIONE-SX (T) // T = Y

NEW T = T → LEFT // NEW T = Z

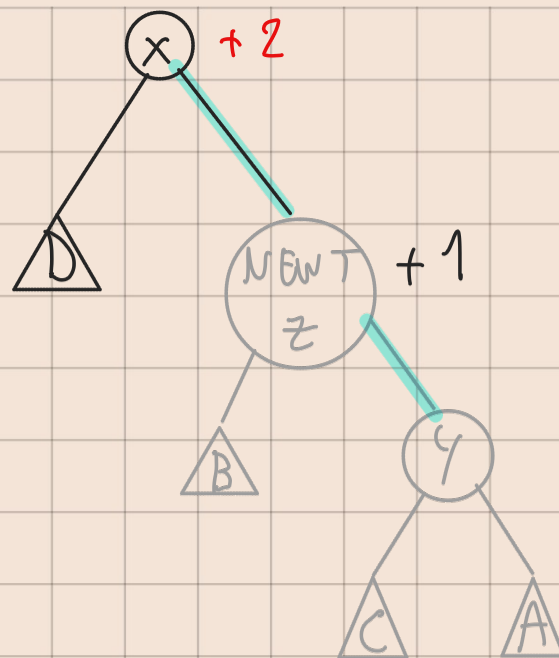
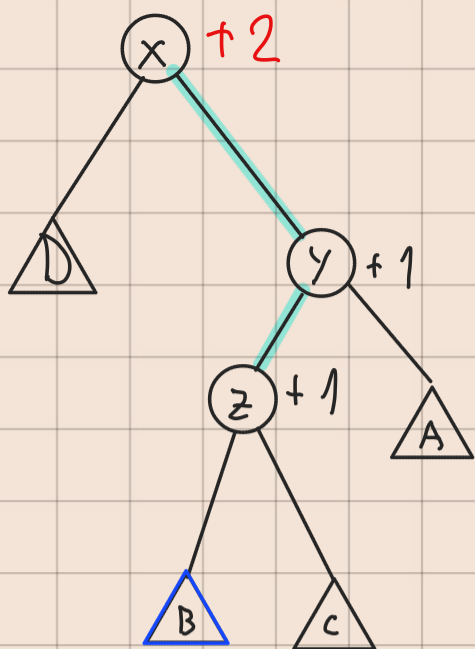
T → LEFT = NEW T → RIGHT // Y → RIGHT = A

NEW T → RIGHT = T // NEW T → RIGHT = Y

T → h = max(ALTEZZA(T → LEFT), ALTEZZA(T → RIGHT)) + 1

NEW T → h = max(ALTEZZA(NEW T → LEFT), ALTEZZA(NEW T → RIGHT)) + 1

RETURN NEW T



2. BILANCIAMENTO: SI ESEGUE UNA ROTAZIONE-DX SULLA RADICE PER ACCORCIARE L'ALTEZZA TOTALE

ROTAZIONE-DX (T) // $T = X$

$NEW T = T \rightarrow RIGHT$ // $NEW T = Z$

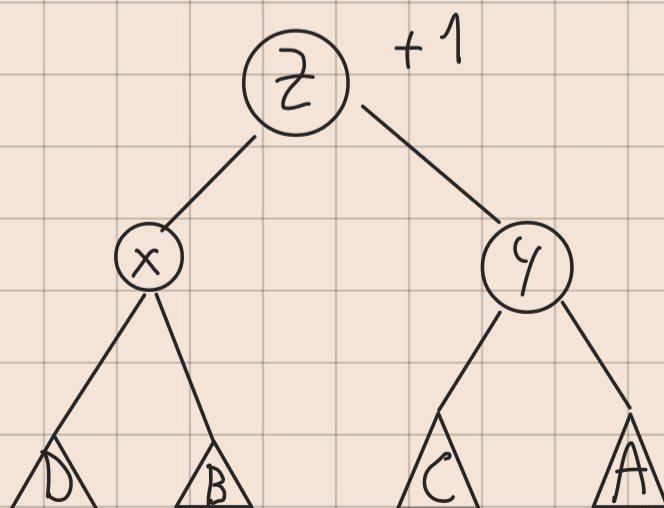
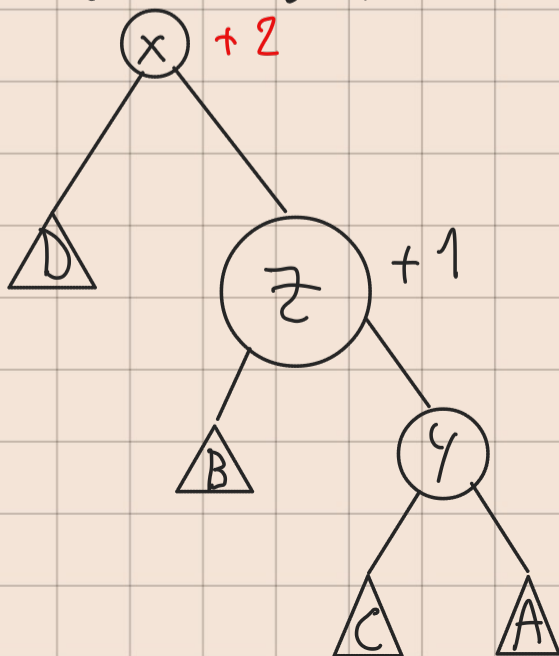
$T \rightarrow RIGHT = NEW T \rightarrow LEFT$ // $X \rightarrow RIGHT = B$

$NEW T \rightarrow LEFT = T$ // $NEW T \rightarrow LEFT = Y$

$T \rightarrow h = \max(\text{altezza}(T \rightarrow LEFT), \text{altezza}(T \rightarrow RIGHT)) + 1$

$NEW T \rightarrow h = \max(\text{altezza}(NEW T \rightarrow LEFT), \text{altezza}(T \rightarrow RIGHT)) + 1$

RETURN NEW T



CANCELLAZIONE IN UN ALBERO AVL

ANALOGAMENTE ALL' INSERIMENTO, ANCHE LA CANCELLAZIONE NECESSITA DI UN' OPERAZIONE DI RIBILANCIAMENTO, DATO CHE LA DIMINUZIONE DELL'ALTEZZA DELL'ALBERO POTREBBE INVALIDARE LA PROPRIETA' FONDAMENTALE DEGLI AVL.

POSSIAMO DIVIDERE LA CANCELLAZIONE IN 3 CASI POSSIBILI:

1. CANCELLAZIONE DI UN NODO FOGLIA
2. CANCELLAZIONE DI UN NODO CON UN SOLO FIGLIO
3. CANCELLAZIONE DI UN NODO CON DUE FIGLI

E' BENE NOTARE CHE LA CANCELLAZIONE POTREBBE CAUSARE UNO SBILANCIAMENTO IN UN NODO DIVERGO DA QUELLO CANCELLATO, PER QUESTO MOTIVO, L'OPERAZIONE DI RIBILANCIAMENTO DELL'ALBERO POTREBBE ESSERE ESEGUITA RICORSIVAMENTE FINO ALLA RADICE DELL'ALBERO

DELETE (T, k)

IF $T \neq \text{NIL}$

IF $T \rightarrow \text{key} > k$

$T \rightarrow \text{LEFT} = \text{DELETE}(T \rightarrow \text{LEFT}, k)$

$T = \text{BILANCIA-DX}(T)$

ELSE IF $T \rightarrow \text{key} < k$

$T \rightarrow \text{RIGHT} = \text{DELETE}(T \rightarrow \text{RIGHT}, k)$

$T = \text{BILANCIA-SX}(T)$

ELSE

$T = \text{DELETE-ROOT}(T)$

RETURN T

DELETE-ROOT (T)

IF $T \neq \text{NIL}$ THEN

 TMP = T

 IF $T \rightarrow \text{LEFT} == \text{NIL}$

 T = T $\rightarrow$ RIGHT

 ELSE IF $T \rightarrow \text{RIGHT} == \text{NIL}$

 T = T $\rightarrow$ LEFT

 ELSE

 TMP = STACCA-MIN (T $\rightarrow$ RIGHT, T)

 T $\rightarrow$ KEY = TMP $\rightarrow$ KEY

 T = BILANCIA-SX (T)

 FREE (TMP)

RETURN T

STACCA-MIN (T, P)^{→ PADRE}

IF $T \neq \text{NIL}$ && $P \neq \text{NIL}$

 IF $T \rightarrow \text{LEFT} \neq \text{NIL}$

 RGT = STACCA-MIN (T $\rightarrow$ LEFT, T)

 NEWT = BILANCIA-DX (T)

 ELSE

 RGT = T

 NEWT = T $\rightarrow$ RIGHT

 IF $T == P \rightarrow \text{LEFT}$

 P $\rightarrow$ LEFT = NEWT

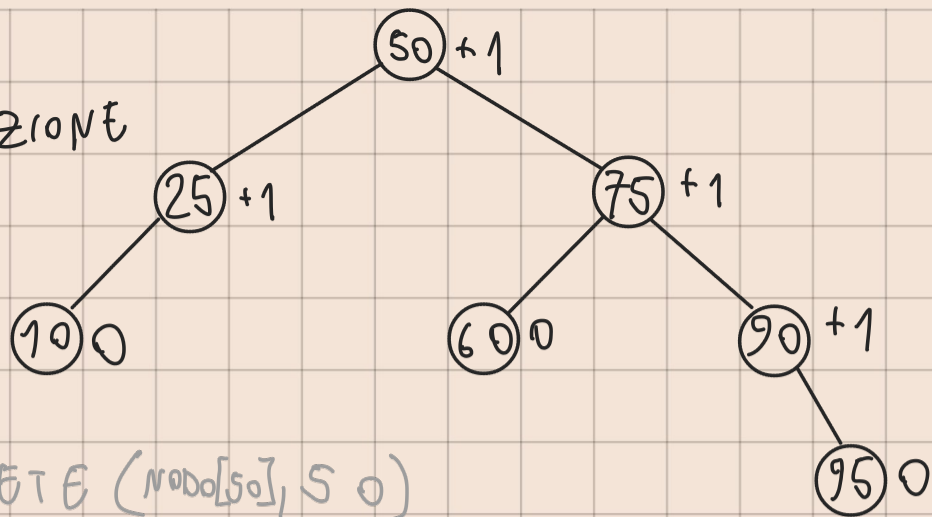
 ELSE

 P $\rightarrow$ RIGHT = NEWT

RETURN RGT

ESEMPIO

EFFETTUIAMO L'OPERAZIONE
DI ELIMINAZIONE DEL
NODO $k=50$



DELETE(T, k) // DELETE(NODO[50], 50)

IF $T \neq \text{NIL}$ // TRUE

IF $T \rightarrow \text{KEY} > k$ // FALSE

$T \rightarrow \text{LEFT} = \text{DELETE}(T \rightarrow \text{LEFT})$

$T = \text{BILANCIA-DX}(T)$

ELSE IF $T \rightarrow \text{KEY} < k$ // FALSE

$T \rightarrow \text{RIGHT} = \text{DELETE}(T \rightarrow \text{RIGHT})$

$T = \text{BILANCIA-SX}(T)$

ELSE

• $T = \text{DELETE-ROOT}(T)$ // DELETE-ROOT(NODO[50])

RETURN T

DELETE-ROOT(T) // $T = \text{NODO}[50]$

IF $T \neq \text{NIL}$ THEN // TRUE

$\text{TMP} = T$ $\text{TMP} = T$

IF $T \rightarrow \text{LEFT} == \text{NIL}$ // FALSE

$T = T \rightarrow \text{RIGHT}$

ELSE IF $T \rightarrow \text{RIGHT} == \text{NIL}$ // FALSE

$T = T \rightarrow \text{LEFT}$

ELSE

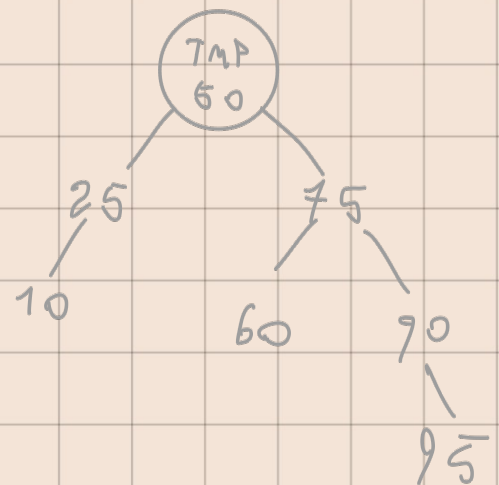
• $\text{TMP} = \text{STACCA-MIN}(T \rightarrow \text{RIGHT}, T)$ // STACCA-MIN(NODO[75], NODO[50])

$T \rightarrow \text{KEY} = \text{TMP} \rightarrow \text{KEY}$

$T = \text{BILANCIA-SX}(T)$

FREE(TMP)

RETURN T



```

1 STACCA-MIN (T, P) // T = NODO[75], P = NODO[50]
  IF T ≠ NIL && P ≠ NIL // TRUE
    IF T → LEFT ≠ NIL // TRUE
      • RET = STACCA-MIN (T → LEFT, T) // STACCA-MIN (NODO[60], NODO[75])
      NEWT = BILANCIA-DX (T)
    ELSE
      RET = T
      NEWT = T → RIGHT
    IF T == P → LEFT
      P → LEFT = NEWT
    ELSE
      P → RIGHT = NEWT
  RETURN RET

```

```

2 STACCA-MIN (T, P) // T = NODO[60], P = NODO[75]
  IF T ≠ NIL && P ≠ NIL // TRUE
    IF T → LEFT ≠ NIL // FALSE
      RET = STACCA-MIN (T → LEFT, T)
      NEWT = BILANCIA-DX (T)
    ELSE
      RET = T // RET = NODO[60]
      NEWT = T → RIGHT // NEWT = NIL
    IF T == P → LEFT // TRUE
      P → LEFT = NEWT // NODO[75] → LEFT = NIL
    ELSE
      P → RIGHT = NEWT
  RETURN RET // RETURN RET [60]

```

RET
GO

75
└ 70
 └ 96

1 STACCA-MIN (T, P) // T = NODO[75], P = NODO[50]

IF T $\neq$ NIL && P $\neq$ NIL // TRUE

IF T $\rightarrow$ LEFT $\neq$ NIL // TRUE

RGT = STACCA-MIN (T $\rightarrow$ LEFT, T) // RGT = RGT[60]

• NEWT = BILANCIA-DX (T) // BILANCIA-DX (NODO[75])

ELSE

RGT = T

NEWT = T $\rightarrow$ RIGHT

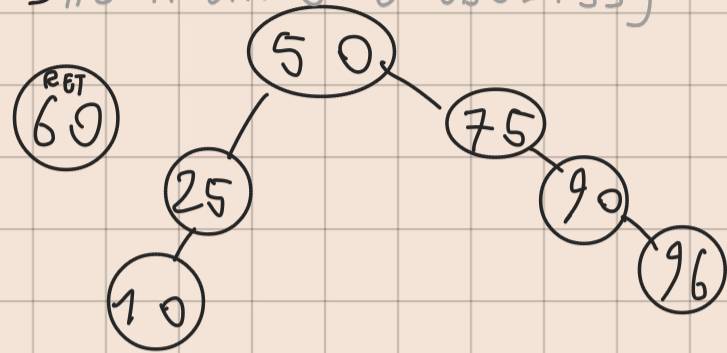
IF T == P $\rightarrow$ LEFT

P $\rightarrow$ LEFT = NEWT

ELSE

P $\rightarrow$ RIGHT = NEWT

RETURN RGT



BILANCIA-DX (T) // T = NODO[75]

IF T $\neq$ NIL // TRUE

IF ALTEZZA (T $\rightarrow$ RIGHT) - ALTEZZA (T $\rightarrow$ LEFT) > 1 // TRUE

DX = T $\rightarrow$ RIGHT // DX = NODO[90]

IF ALTEZZA (DX $\rightarrow$ RIGHT) $\geq$ ALTEZZA (DX $\rightarrow$ LEFT) // TRUE

T = ROTAZIONE-DX (T) // T = NODO[90]

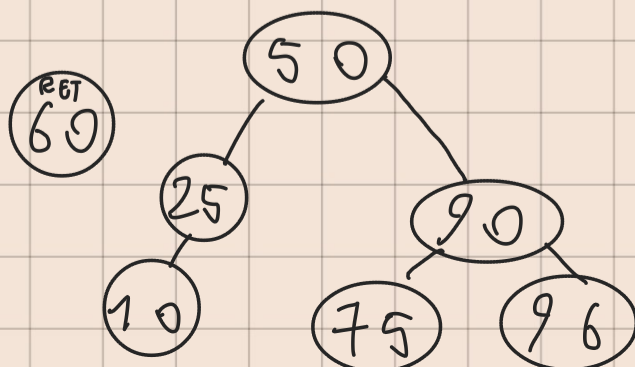
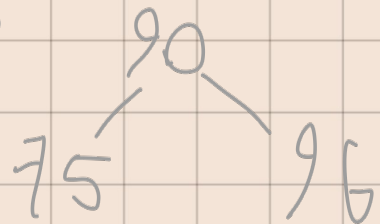
ELSE

T = ROTAZIONE-DOPPIA-DX (T)

ELSE

T $\rightarrow$ h = max (ALTEZZA (T $\rightarrow$ LEFT), ALTEZZA (T $\rightarrow$ RIGHT)) + 1

RETURN T RETURN NODO 90



```

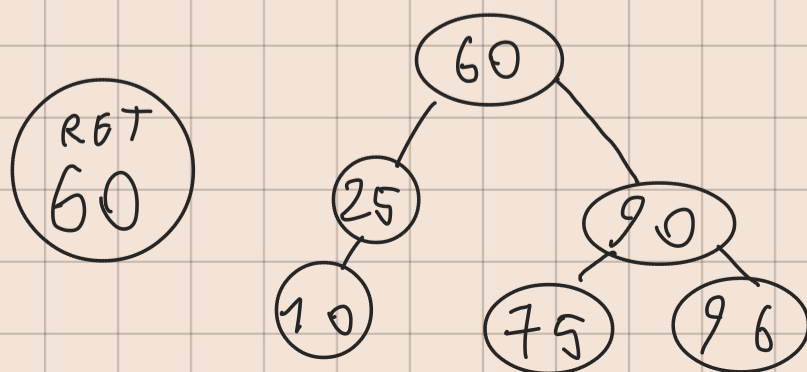
1 STACCA-MIN (T, P) // T = NODO[75], P = NODO[50]
  IF T ≠ NIL && P ≠ NIL // TRUE
    IF T → LEFT ≠ NIL // TRUE
      RGT = STACCA-MIN (T → LEFT, T) // RGT = RGT[60]
      NEWT = BILANCIA-DX (T) // NEWT = NODO[90]
    ELSE
      RGT = T
      NEWT = T → RIGHT
    IF T == P → LEFT // FALSE
      P → LEFT = NEWT
    ELSE
      P → RIGHT = NEWT // NODO[50] → RIGHT = NODO[90]
  RETURN RGT RETURN NODO[60]

```

```

DELETE-ROOT (T) // T = NODO[50]
  IF T ≠ NIL THEN // TRUE
    TMP = T // TMP = T
    IF T → LEFT == NIL // FALSE
      T = T → RIGHT
    ELSE IF T → RIGHT == NIL // FALSE
      T = T → LEFT
    ELSE
      TMP = STACCA-MIN (T → RIGHT, T) // TMP = RGT [60]
      T → KEY = TMP → KEY // NODO[50] → KEY = 60
      • T = BILANCIA-SX (T) // T = BILANCIA-SX (NODO[60])
    FREE (TMP)
  RETURN T

```



BILANCIA-SX (T) // T = NODO[60]

IF T ≠ NIL TRUE

IF ALTEZZA(T → LEFT) - ALTEZZA(T → RIGHT) > 1 FALSE

SX = T → LEFT

IF ALTEZZA(SX → LEFT) ≥ ALTEZZA(SX → RIGHT)

T = ROTAZIONE-SX(T)

ELSE

T = ROTAZIONE-DOPPIA-SX(T)

ELSE

T → h = max(ALTEZZA(T → LEFT), ALTEZZA(T → RIGHT)) + 1

RETURN T // RETURN NODO[60]

NODO[60] → h = 2

DELETE-ROOT(T) // T = NODO[50]

IF T ≠ NIL THEN // TRUE

TMP = T // TMP = T

IF T → LEFT == NIL // FALSE

T = T → RIGHT

ELSE IF T → RIGHT == NIL // FALSE

T = T → LEFT

ELSE

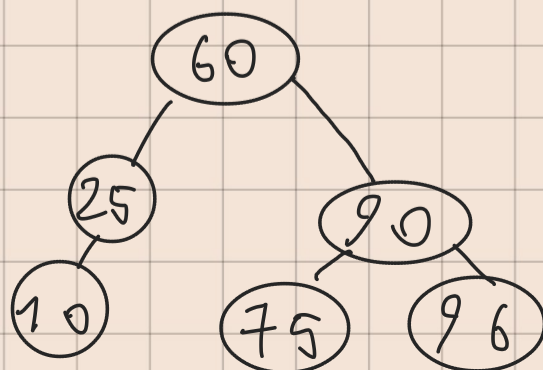
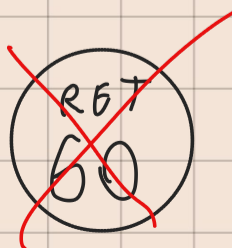
TMP = STACCA-MIN(T → RIGHT, T) // TMP = RET [60]

T → KEY = TMP → KEY // NODO[50] → KEY = 60

T = BILANCIA-SX(T) // T = NODO[60]

• FREE(TMP)

RETURN T



DELETE-ROOT(T) // T = NODO[50]

IF T $\neq$ NIL THEN // TRUE

TMP = T // TMP = T

IF T $\rightarrow$ LEFT == NIL // FALSE

T = T $\rightarrow$ RIGHT

ELSE IF T $\rightarrow$ RIGHT == NIL // FALSE

T = T $\rightarrow$ LEFT

ELSE

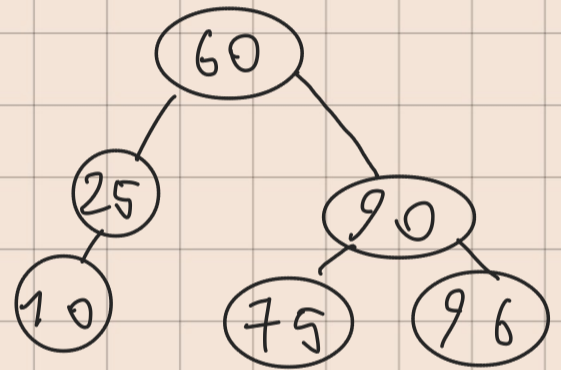
TMP = STACCA-MIN(T $\rightarrow$ RIGHT, T) // TMP = RGT [60]

T $\rightarrow$ KEY = TMP $\rightarrow$ KEY // NODO[50] $\rightarrow$ KEY = 60

T = BILANCIA-SX(T) // T = NODO[60]

FREE(TMP)

RETURN T // RETURN NODO[60]



DELETE(T, K) // DELETE(NODO[50], 50)

IF T $\neq$ NIL // TRUE

IF T $\rightarrow$ KEY > K // FALSE

T $\rightarrow$ LEFT = DELETE(T $\rightarrow$ LEFT)

T = BILANCIA-DX(T)

ELSE IF T $\rightarrow$ KEY < K // FALSE

T $\rightarrow$ RIGHT = DELETE(T $\rightarrow$ RIGHT)

T = BILANCIA-SX(T)

ELSE

T = DELETE-ROOT(T) // T = NODO[60]

RETURN T // RETURN NODO[60]

RESULTATO FINALE